Marketplace Engine

Software design document

Module structure, interfaces and testing strategy

Version 1.0 · 10/09/2026

Repository: bradroberts88/marketplace-engine-core

Product codename: AutoPost

Contents

Companion to `TECHNICAL-DESIGN.md`. That document describes the system; this one describes the software: module boundaries, interfaces, data structures, state machines, and conventions.

1. Repository structure

```
docs/                     Operational and reference documentation
bench/                    Bench launcher and settings template
connector/
  src/                    Agent runtime (agent, tunnel, claim, dashboard, wifi-recovery)
  src/_test/              Pure-Node unit tests for the runtime
  server/                 VPS tunnel server (control WS, proxy, claim, admin)
  flasher/                Electron card-writing app (+ _test/)
  install/                Windows install/uninstall scripts
  keep-alive/             Windows supervisor scheduled task
  deploy/pi/              systemd units, install/verify/self-test scripts, recovery
  deploy/pi/golden/       Golden image build kit
  scripts/sign-build.js   Ed25519 release signing
  build/                  Packaging assets (icon, NSIS hooks)
```

Build outputs (`node_modules`, `.exe`, packaged Electron trees, `.img.xz`) are ignored by git and shipped as release assets.

2. Layering rules

1. **Pure modules** (`flasher/safety.js`, `flasher/inject.js`, `flasher/sha512crypt.js`) have no I/O and no Electron dependency, so they are unit-testable with bare Node. All risky decisions live here.
2. **Process modules** (`flasher/main-flasher.js`, `flasher/writer.js`, `src/agent.js`, `server/src/*`) do I/O and call into the pure modules for every decision that could damage a device or leak traffic.
3. **UI** (`flasher/index.html`, `firstrun-ui.js`, `preview-ui.js`, `dashboard.js`) never touches privileged APIs directly; the only bridge is `flasher/preload.js`, which exposes a fixed set of operations (scan, dry run, flash, verify, progress).
4. The connector server is **standalone** — it does not import from the `production` stack.

3. Connector agent (`connector/src`)

3.1 `agent.js`

Responsibilities: dial the control WS, authenticate, serve relay requests, publish heartbeat, accept remote config, apply signed updates with rollback.

State machine:

```
BOOT → LOAD_CONFIG → CONNECTING → AUTHENTICATING → LIVE → (relaying)
                          ▲                          │
                          └──── backoff reconnect ◄──┘
   LOAD_CONFIG fails / config empty → CLAIM (see claim.js) → LOAD_CONFIG
```

Outputs `heartbeat.json` (monotonic timestamp + link state) consumed by the Windows keep-alive supervisor, `selftest.sh`, and the dashboard.

3.2 `tunnel.js` — egress guard

Single decision function: given a target host/port, either open the connection or refuse. Refuses loopback, private IPv4, link-local, CGNAT (100.64.0.0/10), IPv6 ULA/link-local, and anything that resolves into those ranges. Default is refuse; every new address family must be explicitly allowed.

3.3 `claim.js`

Redeems a one-time setup code against the claim endpoint and writes `config.json` **atomically** (temp file, fsync, rename) so a power cut cannot leave a zero-length config — the failure mode that caused the 2026-08-30 field incident and is now covered by `config-resilience.test.js`.

`config.json` shape (placeholders only in `config.example.json`):

```
{
  "dealershipId": "...",
  "dealershipToken": "...",
  "controlUrl": "wss://.../agent"
}
```

3.4 `dashboard.js`

HTTP server bound to `127.0.0.1` only. Serves tunnel state, public IP/geo/latency probe results and the rep list. Read-only; no control operations.

3.5 `wifi-recovery.js`

Never-strand state machine for a single-radio device: while the unit has no working uplink it raises the `AutoPost-Setup` access point with a captive setup page, accepts new credentials, attempts them, and restores AP mode if they fail. It never tears down the AP before a replacement link is confirmed. Backed by `autopost-wifi-recovery.service` and the BLE path (`autopost-ble-setup.py`) as an alternate channel.

4. Tunnel server (`connector/server`)

Modules and interfaces:

Surface	Bind	Interface
Control WS	`controlBindHost:controlPort`	Agent hello, auth, ping/pong, relay frames, config push
CONNECT proxy	`bindHost:proxyPort` (localhost in prod)	HTTP CONNECT with `proxyAuth` user/pass
Claim endpoint	public	One-time code redemption, issues dealership id + token
Admin API	localhost only	Mint claim codes, inspect dealership liveness

Dealership identity is stored atomically so a crash cannot corrupt the registry. Liveness is enforced on every request, not cached optimistically. Configuration keys are documented in `connector/server/README.md`.

5. Flasher (`connector/flasher`)

```
index.html (renderer)
      | preload.js (contextBridge: scan | dryRun | flash | verify | onProgress)
      ▼
main-flasher.js ── safety.js  (eligibility predicates, pure)
      |          └─ hub.js     (validate pasted claim code, or mint via admin API)
      ▼
writer.js (elevated)  ── re-verifies target ── rpi-imager write+verify ── inject.js (pure)
```

Design rules:

- The writer **re-verifies** the target device after elevation; the renderer's choice is never

trusted across the privilege boundary.

- One confirmation dialog per batch, one write lock per device, one job id per lane.
- The write engine is Raspberry Pi Imager (external binary), not `etcher-sdk`/`@ronomon/direct-io`

— their O_DIRECT external buffers are blocked by Electron 32's V8 sandbox and buffered-IO verify is untrustworthy. Native npm deps are limited to `drivelist` and `@vscode/sudo-prompt`.

- `inject.js` renders every boot-partition file as a pure string function, so the golden-file test

can assert byte-exact output and shell-injection safety.

6. Windows runtime

- `install/install-connector.ps1` registers the hidden `DealershipConnector` scheduled task with

restart-on-failure and a supervisor that relaunches `src/agent.js`.

- `keep-alive/` installs a per-user task (no admin) at logon and every minute, running on battery,

launched invisibly by `run-hidden.vbs`. It restarts AutoPost when the process is missing or when `heartbeat.json` is older than 90 s. The single permitted opt-out is `%LOCALAPPDATA%\AutoPost\disabled.flag`.

- `build/installer.nsh` registers the keep-alive task on install and removes it on uninstall.

7. Pi image build (`connector/deploy/pi/golden`)

Two routes:

1. **customize-stock-image.sh** — the documented shipping path. Takes a stock Raspberry Pi OS Lite

image, appends a data partition, installs the AutoPost stage (services, scripts, WiFi profiles, EEPROM/locale settings), configures partition growth and the read-only overlay, and emits `.img.xz`.

2. **pi-gen** — full chroot build. Disabled unless `ALLOW_STALE_PIGEN_STAGE=1`, because the stage

is not kept current.

Outputs are published as GitHub release assets with SHA256 hashes, never committed.

8. Interfaces summary

From	To	Protocol	Auth
Rep session	Tunnel server	HTTP CONNECT	`proxyAuth` user/pass
Agent	Tunnel server	WSS	`dealershipToken`
First boot	Claim endpoint	HTTPS	One-time claim code
Operator	Local dashboard	HTTP on 127.0.0.1	Host-local only
Operator	Pi	SSH over LAN / USB gadget / Tailscale	Fleet key + password
Release host	Fleet	Signed update bundle	Ed25519, pinned public key

9. Coding conventions

- Node 18+, CommonJS in the connector tree.
- Tests are plain Node scripts with no framework and no install step, so they run on a bench PC.
- Every safety-critical decision is a pure function with a test; process code may only orchestrate.
- Secrets are never committed: `config.json`, `dealerships.json`, `bench-settings.cmd`, signing

keys and installers are all ignored. Example files carry placeholders only.

- Fail closed everywhere: unknown state means refuse, not allow.

10. Testing strategy

Level	What	Where
Unit (pure)	Safety predicates, injection rendering, hashing, WiFi state machine	`flasher/_test`, `src/_test`
Integration (loopback)	Agent + server + client on one host, real egress IP, fail-closed proof	`connector/test-local.js`, `never-drop-test.js`, `firstrun-test.js`
Server	Claim flow, health alerts	`connector/server/test-claim-flow.js`
Script lint	Shell syntax checks on all Pi deployment scripts	`deploy/pi/`
Manual	Ship test plan, day-one checklist, `pi-verify.sh`	`deploy/pi/*.md`

11. Extension points

- New device platform: implement the agent contract (WS hello, heartbeat, relay frames, egress

guard) and reuse the claim flow unchanged.

- New egress policy: extend `tunnel.js` predicates plus their tests; nothing else should need to

change.

- New card layout: add a renderer to `inject.js` and a golden-file case.
- Per-unit secrets: replace the fleet-wide bench settings with values minted by the admin API at

flash time; the injection layer already takes them as parameters.